LAPORAN ALGORITMA DAN STRUKTUR DATA

Analisis Kompleksitas Algoritma

Dibuat oleh:

Nama	**: Fika Yunita**
NIM	**: 2025573010012**
Kelas	**: 1C**
Mata Kuliah	**: Algoritma dan Struktur Data**
Program Studi	**: Teknik Informatika**

Dosen Pengampu:

Muhammad Reza Zulman, S.ST., M.Sc

JURUSAN TEKNOLOGI INFORMASI DAN KOMPUTER

POLITEKNIK NEGERI LHOKSEUMAWE

TAHUN 2026

KATA PENGANTAR

Alhamdulillahirabbil'alamin segala puji dan syukur penulis sampaikan atas kehadirat Allah SWT atas limpahan rahmat, dan hidayah-Nya kepada penulis. Shalawat beriring salam kepada Nabi Besar Muhammad SAW, sehingga penulis dapat menyelesaikan laporan praktikum yang berjudul " Algoritma dan Struktur Data " dengan baik dan tepat waktu. Penulis mengucapkan banyak terimakasih kepada Bapak **Muhammad Reza Zulman, S.ST., M.Sc** telah membimbing penulis dalam menyusun laporan ini.

Laporan ini disusun sebagai bentuk dokumentasi dari kegiatan praktikum yang telah dilaksanakan. Materi dalam modul ini sangat penting karena menjadi dasar dalam memahami efisiensi algoritma sebelum mempelajari struktur data yang lebih kompleks.Dalam Menyusun laporan ini, penulis sangat menyadari atas kekurangan dalam laporan ini baik dari isi maupun struktur penulisannya. Oleh karena itu, kritik dan saran sangat diharapkan untuk perbaikan laporan penulis. Semoga laporan ini dapat memberikan informasi serta menambah wawasan bagi pembaaca dan dapat bermanfaat bagi kita semua.

Lhokseumawe, 25 April 2026

Penulis

DAFTAR ISI

KATA PENGANTAR 2
BAB I 5
PENDAHULUAN 5
1.1 Latar Belakang 5
1.2 Rumusan Masalah 5
1.3 Tujuan Praktikum 5
1.4 Manfaat Praktikum 6
BAB II 7
LANDASAN TEORI 7
2.1 Big O Natation 7
2.2 Time Complexity 7
2.3 Space Complexity 7
2.4 Time-Space Trade Off 7
BAB III 8
HASIL dan PEMBAHAN 8
3.1 Analisis Waktu Eksekusi Algoritma 8
3.1.1 Pembutan File Program 8
3.1.2 Penulisan Kode Program 8
3.1.3 Hasil Output Program 10
3.2 Implementasi dan Klasifikasi Big O Notation 10
3.2.1 Pembutan File 10
3.2.2 Penulisan Kode 11
3.2.3 Output Program 12
3.3 Analisis Space Complexity 12
3.3.1 Pembutan File 12
3.4 Penulisan Kode 13
3.4.1 Output Program 14
3.5 Optimasi Algoritma melalui Refactoring 14
3.5.1 Pembuatan File 14
3.5.2 Penulisan Kode 15

3.5.3 Output Program **15**
3.6 Analisis Tugas Praktikum **16**
3.6.1 Pembuatan File **16**
3.6.2 Penulisan Kode Program **16**
BAB IV **21**
PENUTUP **21**
4.1 Kesimpulan **21**
4.2 Saran **21**
DAFTAR PUSTAKA **22**

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman, efisiensi algoritma menjadi salah satu faktor penting dalam menentukan kualitas suatu program. Algoritma yang tidak efisien dapat menyebabkan waktu eksekusi yang lama dan penggunaan memori yang besar, terutama ketika jumlah data yang diproses semakin banyak.

Untuk mengatasi hal tersebut, diperlukan suatu metode analisis yang dapat mengukur performa algoritma. Salah satu metode yang digunakan adalah Big O Notation, yang menggambarkan bagaimana kompleksitas algoritma bertumbuh terhadap ukuran input. Selain itu, analisis algoritma juga mencakup Time Complexity dan Space Complexity, yang masing-masing mengukur waktu dan memori yang dibutuhkan oleh algoritma.

1.2 Rumusan Masalah

1. Apa itu Big O Notation?
2. Bagaimana cara menganalisis Time Complexity?
3. Bagaimana cara menganalisis Space Complexity?
4. Bagaimana membandingkan efisiensi algoritma?

1.3 Tujuan Praktikum

- Apa itu Big O Notation?
- Bagaimana cara menganalisis Time Complexity?
- Bagaimana cara menganalisis Space Complexity?
- Bagaimana membandingkan efisiensi algoritma?

1.4 Manfaat Praktikum

- Menambah pemahaman tentang efisiensi algoritma
- Membantu memilih algoritma yang tepat
- Menjadi dasar pembelajaran struktur data

BAB II

LANDASAN TEORI

2.1 Big O Natation

Big O Notation adalah notasi matematis yang digunakan untuk menyatakan kompleksitas suatu algoritma berdasarkan pertumbuhan input.
Jenis-jenis kompleksitas:

- $O(1) \rightarrow$ Konstan
- $O(\log n) \rightarrow$ Logaritmik
- $O(n) \rightarrow$ Linear
- $O(n \log n) \rightarrow$ Linearitmik
- $O(n^2) \rightarrow$ Kuadratik
- $O(2^n) \rightarrow$ Eksponensial

2.2 Time Complexity

Time Complexity menunjukkan waktu yang dibutuhkan algoritma dalam menyelesaikan masalah.

2.3 Space Complexity

Space Complexity menunjukkan jumlah memori yang digunakan selama proses eksekusi.

2.4 Time-Space Trade Off

Trade-off adalah kondisi di mana peningkatan kecepatan algoritma dilakukan dengan menambah penggunaan memori.

BAB III

HASIL dan PEMBAHAN

3.1 Analisis Waktu Eksekusi Algoritma

3.1.1 Pembutan File Program

```
Lenovo@DESKTOP-HQ2TFUI MINGW64 ~/2025573010012_Algoritma_dan_struktur_data/praktikum-5 (main)
$ touch 01-intro-kompleksitas.js
```

Penjelasan:

File 01-intro-kompleksitas.js dibuat sebagai media untuk menguji dan membandingkan beberapa algoritma dengan kompleksitas yang berbeda.

3.1.2 Penulisan Kode Program

```
// Fungsi bantu: ukur waktu eksekusi
Windsurf: Refactor | Explain | ×
function ukurWaktu(label, fn) {
    const mulai = Date.now();
    fn();
    const selesai = Date.now();
    console.log(`${label}: ${selesai - mulai} ms`);
}
const N = 100.000; // ukuran data: 100 ribu
// --- Algoritma A: O(n) — loop satu kali ---
Windsurf: Refactor | Explain | ×
function jumlahkanLinear(n) {
    let total = 0;
    for (let i = 1; i <= n; i++) total += i;
    return total;
}
// --- Algoritma B: O(1) — rumus matematika ---
Windsurf: Refactor | Explain | ×
function jumlahkanRumus(n) {
    return n * (n + 1) / 2;
}
// --- Algoritma C: O(n²) — loop bersarang ---
Windsurf: Refactor | Explain | ×
function cariPasangan(arr) {
    const pasangan = [];
    for (let i = 0; i < arr.length; i++) {
        for (let j = i + 1; j < arr.length; j++) {
            if (arr[i] + arr[j] === 10) pasangan.push([arr[i], arr[j]]);
        }
    }
```

```javascript
    return pasangan;
}
const data = Array.from({ length: 5000 }, (_, i) => i);
console.log('=== Perbandingan Waktu Eksekusi ===');
ukurWaktu('O(1)  jumlahkanRumus  ', () => jumlahkanRumus(N));
ukurWaktu('O(n)  jumlahkanLinear ', () => jumlahkanLinear(N));
ukurWaktu('O(n²) cariPasangan    ', () => cariPasangan(data));
console.log('\nHasil sama?', jumlahkanLinear(100) === jumlahkanRumus(100));

console.log('\n=== Latihan-1 ===');
Windsurf: Refactor | Explain | Generate JSDoc | ×
function fnA(n) {
    return n * 2;
}
Windsurf: Refactor | Explain | Generate JSDoc | ×
function fnB(n) {
    for (let i = 0; i < n; i++) {
        for (let j = 0; j < n; j++)
        {

        }
    }
}
Windsurf: Refactor | Explain | Generate JSDoc | ×
function fnC(n) {
    for (let i = 1; i < n; i *= 2) { }
}

function fnD(arr) {
    arr.forEach(x => {
        arr.forEach(y => {
            arr.forEach(z => { });
        });
    });
}
Windsurf: Refactor | Explain | Generate JSDoc | ×
function hitungKompleksitas(n, fn, useArray = false) {
    const start = Date.now();

    if (useArray) {
        const arr = Array.from({ length: n }, (_, i) => i);
        fn(arr);
    } else {
        fn(n);
    }
    const end = Date.now();
    console.log(`Waktu eksekusi (${fn.name}) untuk n=${n}: ${end - start} ms`);
}
const n = 1000;
hitungKompleksitas(n, fnA);
hitungKompleksitas(n, fnB);
hitungKompleksitas(n, fnC);
hitungKompleksitas(n, fnD, true);
```

Penjelasan:

Kode program berisi beberapa fungsi yang merepresentasikan kompleksitas O(1), O(n), dan O(n^2). Selain itu, digunakan fungsi Date.now() untuk mengukur waktu eksekusi masing-masing algoritma. Program juga mencakup bagian latihan untuk menguji fungsi tambahan.

3.1.3 Hasil Output Program

```
$ node 01-intro-kompleksitas.js
=== Perbandingan Waktu Eksekusi ===
O(1)  jumlahkanRumus  : 1 ms
O(n)  jumlahkanLinear : 0 ms
O(n²) cariPasangan    : 67 ms

Hasil sama? true

=== Latihan-1 ===
Waktu eksekusi (fnA) untuk n=1000: 0 ms
Waktu eksekusi (fnB) untuk n=1000: 5 ms
Waktu eksekusi (fnC) untuk n=1000: 0 ms
Waktu eksekusi (fnD) untuk n=1000: 1459 ms
```

Penjelasan:

Output menampilkan perbandingan waktu eksekusi dari beberapa algoritma. Algoritma O(1) dan O(n) memiliki waktu yang sangat cepat, sedangkan $O(n^2)$ lebih lambat karena menggunakan perulangan bersarang. Pada bagian latihan, terlihat bahwa fungsi dengan kompleksitas lebih tinggi membutuhkan waktu eksekusi yang jauh lebih lama.

3.2 Implementasi dan Klasifikasi Big O Notation

3.2.1 Pembutan File

```
Lenovo@DESKTOP-HQ2TFUI MINGW64 ~/2025573010012_Algoritma_dan_struktur_data (main)
$ touch 02-kelas-big-o.js
```

Penjelasan:

File ini dibuat untuk mengimplementasikan berbagai jenis algoritma berdasarkan klasifikasi Big O.

3.2.2 Penulisan Kode

```
Praktikum-5 > JS 02-kelas-big-o.js > ...
function ambilPertama(arr) { return arr[0]; }
function tambahkanItem(arr, item) { arr.push(item); }
function isGenap(n) { return n % 2 === 0; }
Windsurf: Refactor | Explain | Generate JSDoc | ×
function binarySearch(arr, target) {
    let kiri = 0, kanan = arr.length - 1;
    let langkah = 0;
    while (kiri <= kanan) {
        langkah++;
        const tengah = Math.floor((kiri + kanan) / 2);
        if (arr[tengah] === target) {
            console.log(`  Ditemukan di indeks ${tengah} setelah ${langkah} langkah`);
            return tengah;
        }
        if (arr[tengah] < target) kiri = tengah + 1;
        else kanan = tengah - 1;
    }
    return -1;
}
Windsurf: Refactor | Explain | Generate JSDoc | ×
function cariMax(arr) {
    let maks = arr[0];
    for (const x of arr) if (x > maks) maks = x;
    return maks;
}
Windsurf: Refactor | Explain | Generate JSDoc | ×
function bubbleSortNaif(arr) {
    const a = [...arr];
    for (let i = 0; i < a.length; i++)
        for (let j = 0; j < a.length - i - 1; j++)
```

```
            if (a[j] > a[j + 1]) [a[j], a[j + 1]] = [a[j + 1], a[j]];
    return a;
}
Windsurf: Refactor | Explain | Generate JSDoc | ×
function fibRekursif(n) {
    if (n <= 1) return n;
    return fibRekursif(n - 1) + fibRekursif(n - 2); // dua panggilan rekursif
}
console.log('=== O(1) — selalu cepat ===');
console.log(ambilPertama([10, 20, 30, 40, 50])); // 10
console.log(isGenap(42));                        // true

console.log('\n=== O(log n) — Binary Search ===');
const sorted = Array.from({ length: 1_000_000 }, (_, i) => i);
binarySearch(sorted, 731_452); // cari di 1 juta data

console.log('\n=== O(n) — Linear Search ===');
console.log('Max dari 1000 elemen:',
    cariMax(Array.from({ length: 1000 }, () => Math.random() * 1000 | 0)));

console.log('\n=== O(n²) — Bubble Sort (kecil saja) ===');
console.log(bubbleSortNaif([64, 34, 25, 12, 22, 11, 90]));

console.log('\n=== O(2ⁿ) — Fibonacci Rekursif (n kecil!) ===');
for (let i = 0; i <= 10; i++) process.stdout.write(fibRekursif(i) + ' ');
console.log('');
console.log('\nWaktu fib(35) O(2ⁿ):');
let t = Date.now(); fibRekursif(35); console.log(Date.now() - t, 'ms');
console.log('Waktu fib(35) O(n) memoization:');
```

```
const memo = {};
Windsurf: Refactor | Explain | Generate JSDoc | ×
function fibMemo(n) {
    if (n <= 1) return n;
    if (memo[n]) return memo[n];
    return memo[n] = fibMemo(n - 1) + fibMemo(n - 2);
```

Penjelasan:

Kode mencakup beberapa algoritma seperti Binary Search (O(log n)), perulangan linear (O(n)), serta Fibonacci rekursif ($O(2^n)$).

3.2.3 Output Program

```
$ node 02-kelas-big-o.js
=== O(1) — selalu cepat ===
10
true

=== O(log n) — Binary Search ===
  Ditemukan di indeks 731452 setelah 19 langkah

=== O(n) — Linear Search ===
Max dari 1000 elemen: 999

=== O(n²) — Bubble Sort (kecil saja) ===
[
  11, 12, 22, 25,
  34, 64, 90
]

=== O(2ⁿ) — Fibonacci Rekursif (n kecil!) ===
0 1 1 2 3 5 8 13 21 34 55

Waktu fib(35) O(2ⁿ):
320 ms
Waktu fib(35) O(n) memoization:
0 ms
```

Penjelasan:

Output menunjukkan hasil eksekusi masing-masing algoritma. Binary Search memiliki performa yang cepat, sedangkan Fibonacci rekursif membutuhkan waktu lebih lama.

3.3 Analisis Space Complexity

3.3.1 Pembutan File

```
Lenovo@DESKTOP-HQ2TFUI MINGW64 ~/2025573010012_Algoritma_dan_struktur_data/praktikum-5 (main)
$ touch 03-space-complexity.js
```

Penjelasan:

File ini digunakan untuk menganalisis penggunaan memori pada algoritma.

3.4 Penulisan Kode

Praktikum-5 > JS 03-space-complexity.js > ...

```
Windsurf: Refactor | Explain | Generate JSDoc | ×
function jumlahArray(arr) {
    let total = 0;              // hanya 1 variabel tambahan
    for (const x of arr) total += x;
    return total;
}
Windsurf: Refactor | Explain | Generate JSDoc | ×
function duplikasiArray(arr) {
    const baru = [];            // array baru tumbuh seiring arr
    for (const x of arr) baru.push(x * 2);
    return baru;
}
Windsurf: Refactor | Explain | Generate JSDoc | ×
function faktorialRekursif(n) {
    if (n <= 1) return 1;
    return n * faktorialRekursif(n - 1); // n frame di call stack
}
Windsurf: Refactor | Explain | Generate JSDoc | ×
function faktorialIteratif(n) {
    let hasil = 1;
    for (let i = 2; i <= n; i++) hasil *= i; // hanya 2 variabel
    return hasil;
}
const arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
console.log('Jumlah array   :', jumlahArray(arr));
console.log('Duplikasi array:', duplikasiArray(arr));
console.log('Faktorial 10 rekursif :', faktorialRekursif(10));
console.log('Faktorial 10 iteratif :', faktorialIteratif(10));
Windsurf: Refactor | Explain | Generate JSDoc | ×
function hitungUnik(arr) {
```

```
    const seen = new Set(); // Set tumbuh hingga n elemen
      for (const x of arr) seen.add(x);
    return seen.size;
}
const dataAcak = [1,2,3,2,1,4,5,3,6,4,7];
console.log('Elemen unik:', hitungUnik(dataAcak));

console.log('\n=== Latihan-2 ===');
Windsurf: Refactor | Explain | Generate JSDoc | ×
function cariPasanganLambat(arr, target) {
    for (let i = 0; i < arr.length; i++) {
        for (let j = i + 1; j < arr.length; j++) {
            if (arr[i] + arr[j] === target) {
                return [arr[i], arr[j]];
            }
        }
    }
    return null;
}
Windsurf: Refactor | Explain | Generate JSDoc | ×
function cariPasanganCepat(arr, target) {
    const set = new Set();

    for (let i = 0; i < arr.length; i++) {
        const selisih = target - arr[i];
        if (set.has(selisih)) {
            return [selisih, arr[i]];
        }
        set.add(arr[i]);
```

```
    }
    return null;
}
Windsurf: Refactor | Explain | Generate JSDoc | X
function ukurWaktu(fn, arr, target) {
    const start = Date.now();
    const hasil = fn(arr, target);
    const end = Date.now();
    console.log(`${fn.name} -> hasil:`, hasil, `| waktu: ${end - start} ms`);
}

const arrTest = [2, 7, 11, 15];
const targetTest = 9;

ukurWaktu(cariPasanganLambat, arrTest, targetTest);
ukurWaktu(cariPasanganCepat, arrTest, targetTest);
const arrBesar = Array.from({ length: 50000 }, () => Math.floor(Math.random() * 100000));
const targetBesar = -1;

ukurWaktu(cariPasanganLambat, arrBesar, targetBesar);
ukurWaktu(cariPasanganCepat, arrBesar, targetBesar);
```

Penjelasan:

Kode berisi fungsi yang menggunakan variabel sederhana dan struktur data seperti array untuk membandingkan penggunaan memori.

3.4.1 Output Program

```
$ node 03-space-complexity.js
Jumlah array   : 55
Duplikasi array: [
   2,  4,  6,  8, 10,
  12, 14, 16, 18, 20
]
Faktorial 10 rekursif : 3628800
Faktorial 10 iteratif : 3628800
Elemen unik: 7

=== Latihan-2 ===
cariPasanganLambat -> hasil: [ 2, 7 ] | waktu: 1 ms
cariPasanganCepat -> hasil: [ 2, 7 ] | waktu: 0 ms
cariPasanganLambat -> hasil: null | waktu: 3471 ms
cariPasanganCepat -> hasil: null | waktu: 8 ms
```

Penjelasan:

Output menunjukkan perbedaan penggunaan memori antara algoritma sederhana dan algoritma yang menggunakan struktur data tambahan.

3.5 Optimasi Algoritma melalui Refactoring

3.5.1 Pembuatan File

```
Lenovo@DESKTOP-HQ2TFUI MINGW64 ~/2025573010012_Algoritma_dan_struktur_data/praktikum-5 (main)
$ touch 04-refactor.js
```

Penjelasan:

File ini dibuat untuk membandingkan algoritma sebelum dan sesudah dilakukan optimasi.

3.5.2 Penulisan Kode

```
Praktikum-5 > JS 04-refactor.js > ...
Windsurf: Refactor | Explain | Generate JSDoc | ×
function adaDuplikatLambat(arr) {
    for (let i = 0; i < arr.length; i++)
        for (let j = i+1; j < arr.length; j++)
            if (arr[i] === arr[j]) return true;
    return false;
}
Windsurf: Refactor | Explain | Generate JSDoc | ×
function adaDuplikatCepat(arr) {
    const seen = new Set();
    for (const x of arr) {
        if (seen.has(x)) return true;
        seen.add(x);
    }
    return false;
}
Windsurf: Refactor | Explain | Generate JSDoc | ×
function frekuensiLambat(arr) {
    const unik = [];
    const hitung = [];
    for (const x of arr) {
        const idx = unik.indexOf(x);   // O(n) di dalam loop O(n)
        if (idx === -1) { unik.push(x); hitung.push(1); }
        else hitung[idx]++;
    }
    return Object.fromEntries(unik.map((u,i) => [u, hitung[i]]));
}
Windsurf: Refactor | Explain | Generate JSDoc | ×
function frekuensiCepat(arr) {
    const counter = {};
```

```
    for (const x of arr) counter[x] = (counter[x] || 0) + 1;
    return counter;
}
const besar = Array.from({length: 20000}, () => Math.floor(Math.random() * 1000));

let t = Date.now();
adaDuplikatLambat(besar);
console.log('Duplikat LAMBAT O(n²):', Date.now()-t, 'ms');

t = Date.now();
adaDuplikatCepat(besar);
console.log('Duplikat CEPAT  O(n) :', Date.now()-t, 'ms');

const kata = ['a','b','a','c','b','a','d'];
console.log('\nFrekuensi:', frekuensiCepat(kata));
```

Penjelasan:

Kode awal menggunakan nested loop ($O(n^2)$), kemudian dioptimasi menggunakan struktur data seperti Set sehingga menjadi lebih efisien ($O(n)$).

3.5.3 Output Program

```
$ node 04-refactor.js
Duplikat LAMBAT O(n²): 0 ms
Duplikat CEPAT  O(n) : 0 ms

Frekuensi: { a: 3, b: 2, c: 1, d: 1 }
```

Penjelasan:

Output menunjukkan bahwa algoritma yang telah dioptimasi memiliki waktu eksekusi yang lebih cepat dibandingkan algoritma sebelumnya.

3.6 Analisis Tugas Praktikum

3.6.1 Pembuatan File

```
Lenovo@DESKTOP-HQ2TFUI MINGW64 ~/2025573010012_Algoritma_dan_struktur_data/praktikum-5 (main)
$ touch tugas/tugas-1.js tugas/tugas-2.js
```

Penjelasan:

File tugas-1.js dan tugas-2.js dibuat untuk mengerjakan tugas yang diberikan pada modul, yaitu implementasi dan analisis kompleksitas algoritma.

3.6.2 Penulisan Kode Program

a. Tugas-1.js

```
Praktikum-5 > tugas > JS tugas-1.js > ...
// Fungsi A: Intersection dari dua array
// Versi O(n²) menggunakan nested loop
Windsurf: Refactor | Explain | X
function intersectionO_n2(arr1, arr2) {
  let result = [];
  for (let i = 0; i < arr1.length; i++) {
    for (let j = 0; j < arr2.length; j++) {
      if (arr1[i] === arr2[j] && !result.includes(arr1[i])) {
        result.push(arr1[i]);
      }
    }
  }
  return result;
}

// Versi O(n) menggunakan Set
Windsurf: Refactor | Explain | X
function intersectionO_n(arr1, arr2) {
  let set2 = new Set(arr2);
  let result = new Set();

  for (let i = 0; i < arr1.length; i++) {
    if (set2.has(arr1[i])) {
      result.add(arr1[i]);
    }
  }

  return Array.from(result);
}
```

```javascript
// Fungsi B: Kelompok anagram dari array string
// Sortir setiap kata sebagai key
Windsurf: Refactor | Explain | ×
function groupAnagrams(arr) {
  let anagramMap = {};

  for (let word of arr) {
    // Sortir huruf dalam kata untuk dijadikan key
    let sorted = word.split('').sort().join('');

    if (!anagramMap[sorted]) {
      anagramMap[sorted] = [];
    }
    anagramMap[sorted].push(word);
  }

  return Object.values(anagramMap);
}

// Fungsi C: Cek dua elemen yang jumlahnya sama dengan kuadrat elemen ketiga
// Versi O(n³)
Windsurf: Refactor | Explain | ×
function hasPerfectSquareSumO_n3(arr) {
  for (let i = 0; i < arr.length; i++) {
    for (let j = i + 1; j < arr.length; j++) {
      for (let k = 0; k < arr.length; k++) {
        if (k !== i && k !== j) {
          let sum = arr[i] + arr[j];
          let square = arr[k] * arr[k];
          if (sum === square) {
```

```javascript
            return true;
          }
        }
      }
    }
  }
  return false;
}

// Versi O(n log n)
Windsurf: Refactor | Explain | ×
function hasPerfectSquareSumO_nlogn(arr) {
  // Sortir array
  let sorted = arr.sort((a, b) => a - b);

  // Untuk setiap elemen k, cari pasangan i,j dengan i+j = k²
  for (let k of arr) {
    let target = k * k;
    let left = 0, right = sorted.length - 1;

    while (left < right) {
      let sum = sorted[left] + sorted[right];

      if (sum === target) {
        return true;
      } else if (sum < target) {
        left++;
      } else {
```

```javascript
        right--;
      }
    }
  }

  return false;
}

// Test Fungsi A
console.log('=== Fungsi A: Intersection ===');
console.log('O(n²):', intersectionO_n2([1, 2, 3, 4], [3, 4, 5, 6]));
console.log('O(n):', intersectionO_n([1, 2, 3, 4], [3, 4, 5, 6]));

// Test Fungsi B
console.log('\n=== Fungsi B: Group Anagrams ===');
console.log(groupAnagrams(['eat', 'tea', 'tan', 'ate', 'nat', 'bat']));

// Test Fungsi C
console.log('\n=== Fungsi C: Perfect Square Sum ===');
console.log('O(n³):', hasPerfectSquareSumO_n3([1, 2, 3, 4, 5]));
console.log('O(n log n):', hasPerfectSquareSumO_nlogn([1, 2, 3, 4, 5]));
```

Penjelasan:

Kode pada tugas-1.js berisi tiga fungsi utama, yaitu:

- Fungsi A: mencari irisan dua array dengan dua metode ($O(n^2)$ dan $O(n)$ menggunakan Set)
- Fungsi B: mengelompokkan anagram
- Fungsi C: mencari kombinasi bilangan dengan pendekatan $O(n^3)$ dan $O(n \log n)$

Output

```
$ node tugas-1.js
=== Fungsi A: Intersection ===
O(n²): [ 3, 4 ]
O(n): [ 3, 4 ]

=== Fungsi B: Group Anagrams ===
[ [ 'eat', 'tea', 'ate' ], [ 'tan', 'nat' ], [ 'bat' ] ]

=== Fungsi C: Perfect Square Sum ===
O(n³): true
O(n log n): true
```

Penjelasan:

Output menunjukkan bahwa semua fungsi menghasilkan hasil yang sama meskipun menggunakan pendekatan algoritma yang berbeda. Perbedaan utama terletak pada efisiensi waktu eksekusi.

b. Tugas-2.js

```
Praktikum-5 > tugas > JS tugas-2.js > fn_On
Windsurf: Refactor | Explain | Generate JSDoc | ×
function fn_O1(n) {
  return n + 1;
}
Windsurf: Refactor | Explain | Generate JSDoc | ×
function fn_On(n) {
  let sum = 0;
  for (let i = 0; i < n; i++) {
    sum += i;
  }
  return sum;
}
Windsurf: Refactor | Explain | Generate JSDoc | ×
function fn_OnLogn(n) {
  let sum = 0;
  for (let i = 0; i < n; i++) {
    for (let j = 0; j < Math.log2(n); j++) {
      sum++;
    }
  }
  return sum;
}
Windsurf: Refactor | Explain | Generate JSDoc | ×
function fn_On2(n) {
  let sum = 0;
  for (let i = 0; i < n; i++) {
    for (let j = 0; j < n; j++) {
      sum++;
    }
  }
```

```
  return sum;
}
Windsurf: Refactor | Explain | Generate JSDoc | ×
function benchmarkSemua(ukuranData) {
  ukuranData.forEach(size => {
    const data = Array.from({length: size}, (_, i) => i);
    console.log(`Ukuran data: ${size}`);
    console.log(`fn_O1: ${measureExecutionTime(() => fn_O1(size))} ms`);
    console.log(`fn_On: ${measureExecutionTime(() => fn_On(size))} ms`);
    console.log(`fn_OnLogn: ${measureExecutionTime(() => fn_OnLogn(size))} ms`);
    console.log(`fn_On2: ${measureExecutionTime(() => fn_On2(size))} ms`);
    console.log('----------------------');
  });
}
Windsurf: Refactor | Explain | Generate JSDoc | ×
function measureExecutionTime(func) {
  const startTime = performance.now();
  func();
  const endTime = performance.now();
  return endTime - startTime;
}

benchmarkSemua([100, 500, 1000, 5000, 10000]);
```

Penjelasan:

Kode pada tugas-1.js berisi tiga fungsi utama, yaitu:

- Fungsi A: mencari irisan dua array dengan dua metode ($O(n^2)$ dan $O(n)$ menggunakan Set)
- Fungsi B: mengelompokkan anagram
- Fungsi C: mencari kombinasi bilangan dengan pendekatan $O(n^3)$ dan $O(n \log n)$
-

Output:

```
$ node tugas-2.js
fn_O1: 0.030799999999999272 ms
fn_On: 0.07290000000000418 ms
fn_OnLogn: 0.1880999999999986 ms
fn_On2: 0.38520000000000465 ms
-----------------------
Ukuran data: 500
fn_O1: 0.003500000000002501 ms
fn_On: 0.009700000000002262 ms
fn_OnLogn: 0.42379999999999995 ms
fn_On2: 1.2902000000000058 ms
-----------------------
Ukuran data: 1000
fn_O1: 0.0018000000000029104 ms
fn_On: 0.07989999999999498 ms
fn_OnLogn: 0.20479999999999876 ms
fn_On2: 3.113899999999994 ms
-----------------------
Ukuran data: 5000
fn_O1: 0.002400000000001512 ms
fn_On: 0.02240000000000464 ms
fn_OnLogn: 0.3614999999999995 ms
fn_On2: 75.1378 ms
-----------------------
Ukuran data: 10000
fn_O1: 0.0018000000000029104 ms
fn_On: 0.02549999999999386 ms
fn_OnLogn: 0.6610999999999763 ms
fn_On2: 311.6056 ms
-----------------------
```

Penjelasan:

Output benchmark menunjukkan bahwa waktu eksekusi meningkat seiring dengan bertambahnya ukuran data. Algoritma O(1) tetap konstan, sedangkan $O(n^2)$ mengalami peningkatan waktu yang sangat signifikan.

BAB IV

PENUTUP

4.1 Kesimpulan

dapat disimpulkan bahwa Big O Notation merupakan alat yang sangat penting dalam menganalisis efisiensi algoritma. Dengan menggunakan Big O, kita dapat memahami bagaimana performa algoritma akan berubah ketika ukuran data semakin besar.

Time Complexity membantu dalam mengetahui kecepatan algoritma, sedangkan Space Complexity membantu dalam memahami penggunaan memori. Dari hasil percobaan, terlihat bahwa algoritma dengan kompleksitas kecil seperti O(1) dan O(log n) memiliki performa yang jauh lebih baik dibandingkan algoritma dengan kompleksitas besar seperti $O(n^2)$.

Selain itu, penggunaan struktur data yang tepat seperti Set dan Map dapat meningkatkan efisiensi algoritma secara signifikan. Proses refactoring juga terbukti mampu mengubah algoritma yang kurang efisien menjadi lebih optimal.

Dengan demikian, pemahaman terhadap analisis kompleksitas algoritma sangat penting dalam pengembangan program yang efektif dan efisien.

4.2 Saran

Adapun saran yang dapat diberikan adalah sebagai berikut:

Mahasiswa perlu lebih sering berlatih dalam menganalisis kompleksitas algoritma agar lebih terbiasa dalam menentukan efisiensi program.

Dalam pembuatan program, sebaiknya tidak hanya fokus pada hasil akhir, tetapi juga memperhatikan performa algoritma.

Disarankan untuk selalu melakukan pengujian performa terhadap algoritma yang dibuat.

Gunakan struktur data yang sesuai agar program lebih optimal.

DAFTAR PUSTAKA

Zulman, Muhammad Reza. Modul Praktikum Struktur Data dan Algoritma. 2026.

JavaScript Guide dan Algorithm Concepts. https://developer.mozilla.org

Time Complexity dan Big O Notation. https://www.geeksforgeeks.org